Reversible Planning

Tower of Hanoi

Pietro Giola

February 26, 2026

1 Introduction

AI Planning is a field of Artificial Intelligence which explores the process of using autonomous techniques to solve planning and scheduling problems. A planning problem is one in which we have some initial starting state, which we wish to transform into a desired goal state through the application of a set of actions.

A relevant planning property is reversibility: an action a is uniformly reversible if a single sequence of actions "undoes" all the effects of action a, for every state that a is applicable. Reversibility enables safe exploration strategies in autonomous agent, that means that no state is a dead end.

This technical report presents the design and verification of a reversible STRIPS planning domain, formalized using state-of-the-art automated reasoning systems. The benchmark problem chosen is the classic **Tower of Hanoi** puzzle problem, which provides a clean and well-understood structure and is rich enough to demonstrate the concepts.

The project was carried out through three main steps:

1. Design of the reversible PDDL domain and its grounded encoding, definition of the problem instance, translation with `plasp`, and verification with `eclingo`.
2. Formal verification of reversibility using `qasp`, generating the quantified encoding via `pddl-to-qasp-uurev.py`.
3. Plan generation with `DLV` using the action-language encoding, and constructive computation of reverse plans using the **RevPlan** system.

All source files referenced throughout this report are submitted alongside it.

2 The Tower of Hanoi

2.1 Problem Description

The Tower of Hanoi puzzle is made up of three pegs and a set of disks of varying sizes. The puzzle begins with all disks placed on the first pole in descending order, with the largest disk at the bottom. The aim of the puzzle is to move the disks to the target pole.

2.2 Rules

1. Only **one disk** may be moved at a time.
2. Each move consists of taking the **uppermost** disk from one peg or disk and placing it on top of another peg or disk.
3. No disk may be placed **on top of a smaller disk**.

2.3 STRIPS Representation

For simplicity's sake, the domino will use two discs and three pegs instead of the original problem with 3 disks and three pegs. The domain is scalable, so it is not a problem to tackle a simpler domain.

- **Fluents:** `on(D,L)` disk `D` is on location `L`; `clear(L)` nothing is on top of location `L`.
- **Actions:** moving a disk from one location to another, subject to the legality constraints above.
- **Initial state:** `d2` on `peg1`, `d1` on `d2`, `peg2` and `peg3` clear.
- **Goal:** `d2` on `peg3`, `d1` on `d2`.

3 Step 1: PDDL Encoding and Verification with eclingo

3.1 Tools

PDDL (Planning Domain Definition Language) is the standard language for classical planning. It separates the *domain* (predicates and actions) from the *problem* (objects, initial state, goal). The `:strips` and `:negative-preconditions` requirements are used here.

plasp translates PDDL files into ASP facts.

eclingo is an epistemic extension of `clingo` that supports knowledge operators of the form `&k{}`. It reasons about what holds across *all* possible world views, making it suitable for verifying properties that must hold universally.

3.2 Domain Design

The **lifted domain** (`domain.pddl`) uses typed parameters and is compact and general.

The **grounded domain** (`grounded_domain.pddl`) fully instantiates all actions for the specific objects. Negative preconditions are made explicit (e.g., `not (on_d1_peg2)`) to prevent the solver from reaching physically invalid states. This is the version used throughout the pipeline.

The crucial design point is **reversibility by construction**: for each forward step, an explicit reverse step is added whose preconditions correspond to the post-state of the forward step and whose effects undo the original state. For instance:

- `move_d1_from_peg1_to_peg2`: preconditions include `on_d1_peg1`, `clear_peg2`; effects set `on_d1_peg2` and `clear_peg1`.
- `move_d1_from_peg2_to_peg1`: the symmetric inverse, with swapped source and destination.

3.3 Problem File: `problem.pddl`

The problem file defines the specific scenario:

- **Initial state:** `on_d2_peg1`, `on_d1_d2`, `clear_d1`, `clear_peg2`, `clear_peg3`.
- **Goal:** `on_d2_peg3`, `on_d1_d2`, `clear_d1`, `clear_peg1`, `clear_peg2`.

3.4 Generating the ASP Instance

Domain and problem are translated into a single ASP instance using `plasp`:

```
plasp translate grounded_domain.pddl problem.pddl > instance.lp
```

The output `instance.lp` represents the complete planning problem as logic program facts: domain variables with their Boolean domains, action schemas with preconditions and postconditions, and initial state and goal.

3.5 Verification with eclingo

The grounded domain (without problem) is combined with the sequential-horizon encoding to verify the instance under epistemic semantics:.

```
cat grounded_domain.lp sequential-horizon_uurev.lp | eclingo -
```

The `sequential-horizon_uurev.lp` file encodes the reversibility check as an epistemic logic program. It guesses one action (`chosen/1`), restricts reasoning to fluents relevant to that action (`relevant/1`), and enforces via the operator `&k{∼noreversal}` that no epistemic world view is consistent with a failed reversal.

A *piece* of the output obtained is:

```
Solving...
World view:  1
chosen("move_d1_from_d2_to_peg1")
plan("move_d1_from_peg1_to_d2",1)
SATISFIABLE
```

The result **SATISFIABLE** confirms that the grounded domain is consistent and that reverse plans exist at horizon 1. The world view shows that `move_d1_from_d2_to_peg1` is reversible by `move_d1_from_peg1_to_d2`, exactly as designed.

4 Step 2: Reversibility Verification with qasp

4.1 Tools

qasp (Quantified Answer Set Programming) extends ASP with quantifiers over answer sets. It enables problems of the form "there *exists* a plan such that *for all* initial states satisfying the preconditions, the plan reverses the chosen action". Thanks to this quantified structure we can capture the reversibility property.

The `.qasp` encoding follows a three-block structure:

- **%@exists**: guess a single action to check (`chosen/1`) and a candidate reverse plan (`occurs/2`).
- **%@forall**: for all initial states consistent with the chosen action's preconditions, evaluate the plan using the domain transition semantics embedded from the `plasp` output.
- **%@constraint**: enforce that the plan restores the state exactly to what it was before the chosen action was applied.

4.2 Generating the qasp File

The script `pddl-to-qasp-uurev.py` generates a complete `.qasp` file by calling `plasp translate` internally and embedding the output inside the quantified encoding:

```
python pddl-to-qasp-uurev.py grounded_domain.pddl N > check.qasp
```

The integer argument N defines the **horizon**: the number of reverse steps allowed. The generated file (`check1.qasp` for $N = 1$) will contain time facts, the guessing rules, the `plasp` encoding of the domain, and the reversibility constraints.

4.3 Running the Check

```
java -jar qasp.jar check.qasp
```

The check was run with two horizon values:

- $N = 1$: each action must be directly reversible by a single step. Result: **SAT**.
- $N = 3$: allows reverse plans in three steps. Result: **SAT**.

Both of these results verify that `grounded_domain.pddl` is a reversible STRIPS domain, as there is a reverse plan for every action and every applicable initial state within the horizon. A result of **UNSAT** would indicate that there is at least one action that cannot be reversed.

5 Step 3: Plan Generation with DLV and Reverse Plans

5.1 Tools

DLV (Disjunctive Logic programming with Verification) is a logic programming system supporting disjunctive rules and strong negation. It is used here both as a plan generator (via the action language encoding) and as the underlying solver for RevPlan.

RevPlan is a java front-ended tool for computing *reverse plans* in action languages. Given a domain (`.plan`), domain constants (`.dl`), and an XML conditions file specifying a pre-condition ϕ and a post-condition ψ, RevPlan enumerates all action sequences (within a given horizon) that, starting from a state satisfying ψ, lead back to a state satisfying ϕ.

5.2 Action Language Encoding

The domain is re-encoded in the action language accepted by DLV (`hanoi.plan`), paired with domain constants (`hanoi.dl`):

- `hanoi.plan` declares fluents (`on/2`, `clear/1`), the action `move(D,L,L1)`, executability conditions, causal laws (inertia, caused effects), integrity constraints (`forbidden`), and the initial state and goal.
- `hanoi.dl` provides the ground facts `disk(d1)`, `disk(d2)`, `location(peg1..peg3)`, and `location(D) :- disk(D).`

Running DLV with a horizon of 7 produces the following plan trace:

```
STATE 0:  clear(d1), clear(peg2), clear(peg3), on(d1,d2), on(d2,peg1)
ACTIONS: move(d1,d2,peg3)
STATE 1:  on(d2,peg1), clear(d1), on(d1,peg3), clear(d2), ...
...
STATE 7:  on(d1,d2), on(d2,peg3), clear(d1), clear(peg1), clear(peg2)
PLAN: move(d1,d2,peg3); move(d1,peg3,d2); move(d1,d2,peg3);
      move(d1,peg3,d2); move(d1,d2,peg2);
      move(d2,peg1,peg3); move(d1,peg2,d2)
```

5.3 Reverse Plan Computation with RevPlan

RevPlan requires three input files: `hanoi.plan`, `hanoi.dl`, and `cond.xml`. The conditions file specifies:

- ϕ (pre-condition, `<phi>`): the initial state `on(d1,d2)`, `on(d2,peg1)`, `clear(d1)`, `clear(peg2)`, `clear(peg3)`.
- ψ (post-condition, `<psi>`): the goal state `on(d1,d2)`, `on(d2,peg3)`, `clear(d1)`, `clear(peg1)`, `clear(peg2)`.

RevPlan seeks plans that, starting from any state satisfying ψ, lead back to a state satisfying ϕ. The command to run:

```
java -cp ./plan-library-binary.jar planlibrary.ReverseDomain \
-x hanoi.plan hanoi.dl hanoi_conditions.xml ../../DLV 3
```

Internally, RevPlan produces two auxiliary plan files: `hanoi_rev.plan` (containing the RevPlan machinery barrier fluents `bar_on`, `bar_clear`, a separator action `sep`, and causal laws expressing the snapshot to revert to) and `hanoi_simrev`.

6 Conclusion

This project demonstrated a complete pipeline for encoding, designing, and formally verifying a reversible STRIPS planning domain, using, as a case of study, the 2-disk Tower of Hanoi.

In order to ensure reversibility by design, the grounded domain `grounded_domain.pddl` was created with explicit inverse actions. Consistency and the presence of reverse plans at horizon 1 were confirmed by comparing the translation of `plasp` with `eclingo`. The domain is officially confirmed to be fully reversible when the `qasp` reversibility check returned **SAT** at both horizon 1 and horizon 3. Ultimately, `RevPlan` constructively calculated reverse plans from the goal state back to the initial state, and `DLV` produced a legitimate seven-step plan for the original problem.

The utilized tools `plasp`, `eclingo`, `qasp`, `DLV`, and `RevPlan` offer various guarantees and perspectives on reversibility and are complementary approaches to automated reasoning about planning domains.